PROJECT SYNOPSIS — UrjaNiyam

An Intelligent Constraint-Satisfaction-Based Load-Shedding Scheduler for Power Grids

Submitted by: Parth Sharma (240574), Mitali Salhotra (240538), Mehak Bhardwaj (240535)

1. INTRODUCTION

Electricity distribution companies (DISCOMs) in India routinely face situations in which available supply falls short of demand, especially during summer heatwaves, festival-season spikes, or equipment failure. When this happens, utilities deliberately cut power to selected areas to maintain system stability and prevent more severe supply disruptions, a practice known as load shedding. The National Load Dispatch Centre's Short-Term National Resource Adequacy Plan projected India's peak electricity demand to reach roughly 273 GW in June 2025, against which an unmet demand of 15–20 GW was anticipated during high-risk months, underlining that load shedding is not a rare emergency measure but a recurring operational reality [1].

In most Indian distribution utilities today, the decision of which feeders to cut, for how long, and in what order is made manually and reactively by control-room operators under time pressure, using rotational schedules that are rarely optimised or audited [2], [3]. This creates two persistent failure modes: certain localities — mostly from lower-income or informally settled areas — are shed disproportionately more often, and safety-critical loads are occasionally cut in error; and because shedding is performed through discrete load blocks and under uncertain, rapidly changing operating conditions, the amount disconnected need not always match the shortfall exactly. Power-system research identifies this as over-shedding: disconnecting more load than necessary to resolve the disturbance, thereby imposing avoidable disruption on consumers [3], [4], [17].

This project, UrjaNiyam (Hindi: उर्जा नियम, “energy discipline/regulation”), builds the decision layer that is largely absent from both academic and commercial systems: given a shortfall and a description of a grid's feeders, it computes a fair, safe, and auditable cut schedule using Constraint Satisfaction Problem (CSP) techniques, with a strictly read-only Large Language Model (LLM) layer added only to explain the schedule in plain language.

2. PROBLEM STATEMENT

Given a predicted power shortfall S_t at each time slot over a planning horizon T, UrjaNiyam must decide which feeders to disconnect, when, and for how long.

Each feeder has a time-varying load, operational priority, information about the loads it serves, and a history of previous shedding. Since multiple feeder combinations may meet the same shortfall with very different consequences, the system seeks a schedule that:

- meets the required shortfall;
- respects priority and protection policies;
- limits consecutive downtime and unnecessary switching;
- avoids repeatedly shedding the same eligible feeders; and
- minimises unnecessary load shedding and overall disruption.

Each feeder–time pair is represented as a binary decision: $X_{f,t} \in \{ON, OFF\}$.

The constraints define which schedules are allowed, while the optimisation objective selects the least harmful valid schedule. Fairness is used as a guardrail to prevent repeated concentration of outages, rather than requiring equal treatment across feeders. UrjaNiyam is therefore formulated as a finite-domain CSP with an optimisation objective.

Scope note: S_t is treated strictly as an input to the scheduler, not something UrjaNiyam predicts — forecasting is outside the current scope (see Section 10). The system uses an aggregated feeder-level model grounded in public data; it does not model full electrical behaviour such as power flow, voltage stability, transformer limits, or live SCADA control. Feeder states and shedding histories are simulated. The present project evaluates the scheduling problem in simulation, with real-grid integration treated as a future direction.

3. OBJECTIVES

- To formally model feeder-level load shedding over time as a finite-domain CSP, where each feeder–time pair is a binary ON/OFF decision subject to operational constraints.
- To implement backtracking with arc-consistency (AC-3) and min-conflicts local search from scratch, and compare them with a greedy baseline and a deliberately simplified A* single-slot baseline.
- To design a constraint-relaxation policy for infeasible shortfalls that relaxes only selected soft constraints in a predefined order, while never violating the system's highest-priority protection rules.
- To implement rolling-horizon replanning, allowing the schedule to respond to changing shortfalls while using previous shedding history to control consecutive and repeated outages.

- To define and evaluate an optimisation objective that minimises overall disruption and unnecessary excess shedding, while accounting for load priority, outage duration, switching, and repeated disruption.
- To use fairness as a guardrail against systematically concentrating outages on the same eligible feeders, and evaluate fairness alongside harm and solution quality rather than treating equal outage time as the objective.
- To evaluate the system on a synthetic-but-grounded distribution grid, using public grid and demand data to calibrate the simulation and measuring feasibility, excess shedding, solution quality, solver performance, fairness, scalability, relaxation behaviour, and replanning stability.
- To develop a read-only, evidence-grounded LLM explanation layer that explains scheduling decisions from the solver's constraint trace without influencing the schedule itself.
- To integrate an existing, pre-built forecasting model as a pluggable input adapter for the shortfall signal, so the scheduler can also be demonstrated end-to-end on live-style demand data without the project taking on forecasting research as a core deliverable.

4. LITERATURE REVIEW

The literature relevant to UrjaNiyam spans five threads: optimisation-based load shedding, constraint-satisfaction and local-search scheduling, fairness- and equity-aware curtailment, rolling-horizon control, and grounded LLM explanation, alongside a survey of what utilities and commercial platforms actually deploy today. Rather than reviewing each thread as an isolated silo, the observations below (Section 4.1) are organised around the cross-cutting patterns that recur across them; the resulting gaps are summarised at the end of the section (Section 4.2) and carried forward into Section 5.

4.1 Observations and Patterns

Optimisation formulations solve electrical feasibility, not explainability. Most academic work on load shedding formulates it as an optimal power-flow variant. Bienstock and Verma frame worst-case load shedding as a mixed-integer non-linear program (MINLP) coupled through the AC power-flow equations, showing that even finding a feasible point is computationally hard in general [4]. Follow-on work on AC-constrained optimal load shedding (AC-OLS) similarly formulates the problem as non-convex and NP-hard, turning to learning-based methods to make it tractable at scale [5]. This line of work achieves high electrical fidelity, but the resulting schedules are numerically opaque — a control-room operator cannot easily be shown why a particular bus was shed — and fairness across consumers is rarely a first-class objective.

CSP and local search scale well, but have not met a live, rolling shortfall. Constraint satisfaction techniques have a long history in large-scale scheduling. Minton, Johnston, Philips, and Laird's min-conflicts heuristic repair method solves large CSP and scheduling instances —

including real NASA Hubble Space Telescope observation scheduling — far more efficiently than systematic backtracking on the same class of problem [6]. Constraint-logic-programming approaches have also been applied directly to power-system maintenance scheduling, such as a documented case study of scheduling preventive maintenance across a Belgian utility's generating units [7]. These results support treating load-shedding decisions as a CSP rather than a purely numerical optimisation problem, but none of the reviewed work reasons over a live, continuously updating shortfall the way a rolling-horizon replanning loop requires.

Fairness is emerging as a measurable objective, but as a single-shot term, not a rolling audit. An equity-aware load-shedding optimisation model introduces a grid Gini coefficient as a curtailment-equity indicator, showing that without an explicit equity term specific buses face exponentially higher shedding risk after contingencies, while incorporating equity distributes curtailment more evenly at only a marginal efficiency cost [8]. This is corroborated by real-world reporting on Indian rotational load shedding, where informal settlements and slum clusters have been documented as bearing power cuts disproportionately, prompting explicit government directives that shedding must be strictly rotational across areas [3]. In both cases, fairness is computed once per optimisation run rather than tracked as a rolling, auditable guardrail across successive replanning cycles.

Rolling-horizon control is standard in microgrid dispatch, not yet in feeder-level shedding. Model Predictive Control (MPC), also called receding- or rolling-horizon control, solves a finite-horizon optimisation at each time step, applies only the first control action, and re-solves as new information arrives. This pattern is well established for microgrid energy management — both in simulation and in hardware-validated experimental deployments — where a rolling MILP formulation periodically re-solves dispatch as new load and generation data becomes available [9], [10]. The same control pattern has not, to the reviewed literature's knowledge, been applied to feeder-level shedding decisions with a downtime ledger substituting for conventional state feedback.

Grounding mitigates LLM hallucination, but must be verified empirically, not assumed. A well-documented failure mode of large language models is hallucination — the confident generation of content unsupported by any evidence. Retrieval-Augmented Generation (RAG) is the standard mitigation, substantially reducing unsupported claims in structured-output and question-answering settings [11], [12]. Benchmarks such as RAGTruth show that even grounded systems can still produce partially unsupported text, meaning grounding must be tested empirically rather than assumed [13].

What is actually deployed sits well behind what the research suggests is possible. Manual rotational schedules, greedy SCADA heuristics, and proprietary DERMS/ADMS platforms remain the operational norm (Table 1); none combine the optimisation rigour, fairness accounting, and explainability that the reviewed academic threads each achieve individually.

Approach	How it is used today	Limitation
Manual rotational schedules	Discoms publish fixed rotational cut schedules, adjusted manually during crises	No optimisation or fairness guarantee; slow to adapt to real-time shortfall changes
Pure ML forecasting	Predicts demand/shortfall via regression or ensemble models	Stops at prediction; leaves the harder allocation decision to a human
Greedy heuristic shedding	Cuts the largest non-critical loads first until resolved	Ignores fairness over time; can over-target a few large feeders
Commercial DERMS/ADMS	Some platforms use optimisation internally	Proprietary, expensive, and not explainable or auditable
Academic CSP-scheduling literature	Exam timetabling, hospital bed allocation, crew scheduling	Assumes a fixed resource set over a fixed horizon; none require rolling-horizon replanning against a live, updating shortfall

Table 1. Existing approaches to the load-shedding decision problem and their limitations.

4.2 Gaps Identified

- No reviewed system pairs a transparent, from-scratch CSP solver with a rolling-horizon replanning loop against a live, updating shortfall.
- No reviewed system pairs a stated, tested constraint-relaxation ordering with a guarantee that critical-load protection is never relaxed.
- Fairness is treated in the literature as a single-shot optimisation term, not a rolling, auditable guardrail metric computed every replan cycle.
- Over-shedding — excess disconnection beyond the required cut under discrete, block-wise curtailment — is not treated as a measured optimisation objective in its own right anywhere in the reviewed work.
- No reviewed decision layer is paired with a strictly read-only, boundary-tested LLM explanation layer.

These five gaps, taken together, define the specific contribution developed in Section 5.

5. RESEARCH GAP

Section 4.2 summarises five gaps found across the reviewed literature. Read together, they describe two camps that do not overlap: numerically powerful but opaque optimisation formulations that solve electrical feasibility but not explainability or fairness-over-time, and CSP/local-search techniques with a long scheduling pedigree that have not been applied to a live, rolling-horizon load-shedding decision with an explicit constraint-relaxation cascade (Section 4.1). Forecasting a demand shortfall is itself a well-studied, separate problem; deciding what to do about a known shortfall — fairly, safely, and automatically — is not commonly built, particularly not with an auditable, constraint-based reasoning trail. No reviewed system combines: (i) a

transparent, from-scratch CSP solver, (ii) a stated and tested relaxation ordering for infeasible shortfalls, (iii) a rolling-horizon control loop with a live downtime ledger, (iv) over-shedding treated as a measured optimisation objective rather than an incidental side effect of a hard feasibility constraint, and (v) a strictly read-only, evidence-grounded natural-language explanation layer. This is the specific gap UrjaNiyam is built to close.

6. PROPOSED METHODOLOGY

6.1 System architecture

UrjaNiyam is organised as a closed control loop rather than a single-shot pipeline. A Grid State Store holds feeders, topology, load history, the live shortfall, and the downtime ledger as the single append-only source of truth. A declarative Policy Store holds priority and protection rules, maximum-downtime and switching-frequency limits, fairness-guardrail weights, and harm-cost weights — including a penalty for excess shedding beyond the required cut — editable without touching code. A Constraint Compiler combines state and policy into a CSP instance — variables state(feeder, time-slot) ∈ {ON, OFF}. A Solver Orchestration Layer runs backtracking + AC-3 and min-conflicts (with an A* single-slot baseline for comparison) and arbitrates between them; on infeasibility it applies the relaxation cascade, never touching the priority/protection rules that keep critical loads online. The resulting schedule drives an Actuation Layer (simulated cut/restore commands), an Evaluation Layer (fairness, excess shedding, harm-cost, audit trail), and a read-only LLM Explanation Layer that answers operator queries strictly from the audit trail.

UrjaNiyam control loop — Grid State Store → Constraint Compiler → Solver Orchestration → Schedule → Actuation / Evaluation → LLM Explanation, replanning on a fixed cadence or shortfall-delta trigger.

The system replans on a fixed cadence (e.g., every 5–15 simulated minutes) or whenever the shortfall changes by more than a threshold. Each replan reads the current downtime ledger, so a feeder just cut is not immediately eligible to be cut again unless the constraint set allows it — this is what makes the fairness guardrail and the avoidance of repeated shedding on the same feeders real rather than theoretical.

The shortfall signal S_t that feeds the Grid State Store is produced by a pluggable Forecast Adapter rather than being wired to a single fixed source. For controlled benchmarking (Section 9), the adapter plays back scripted mild/moderate/crisis scenarios so solver comparisons stay reproducible. For an end-to-end, live-style demo, the same adapter can instead be pointed at an existing, pre-built forecasting model run over public Indian demand data, so the project demonstrates a complete pipeline from raw demand signal to auditable cut schedule without UrjaNiyam itself taking on forecasting as a research contribution.

6.2 Agent framing (PEAS)

- Performance measure: minimise weighted harm — disruption × priority × duration, plus a penalty for excess shedding beyond the required cut — subject to priority/protection constraints, with fairness enforced as a guardrail rather than a hard bound.
- Environment: the grid's current state, live shortfall size, and feeder history.
- Actuators: cut/restore commands issued per feeder per time slot.
- Sensors: shortfall magnitude, feeder states, and elapsed downtime per feeder.

6.3 CSP formulation and solvers

Variables: state(feeder_i, time-slot_j) ∈ {ON, OFF}. Constraints: meeting the required shortfall per slot, priority/protection policies for critical loads, maximum consecutive downtime, and switching-frequency limits; fairness and excess-shedding minimisation enter through the optimisation objective rather than as hard bounds, consistent with the Section 2 formulation of UrjaNiyam as a finite-domain CSP with an optimisation objective. Three solving strategies are implemented from scratch (not via an off-the-shelf CSP library) and compared: backtracking search with forward checking and arc-consistency (AC-3); min-conflicts local search; and a greedy baseline for contrast. An A* search over a deliberately simplified single-time-slot formulation (heuristic: remaining shortfall ÷ largest available sheddable load) provides an informed-vs-uninformed search comparison and a sanity-check lower bound — it is reported explicitly as non-comparable to the full CSP, since duration, downtime, switching discipline, and the disruption-minimising objective are not expressible in a single-slot formulation.

6.4 Constraint-relaxation cascade and knowledge representation

When a shortfall is too large to resolve without violating a soft constraint, the system relaxes soft constraints in a fixed, declaratively encoded order — first maximum-downtime limits, then switching-frequency limits, then non-critical priority weighting — and never the priority/protection rules that keep critical loads (hospitals, water pumping, emergency services) online. Fairness is not part of this relaxation ladder: since it is carried as a guardrail inside the optimisation objective rather than enforced as a hard bound, there is nothing to relax — it simply degrades gracefully alongside the objective value as pressure increases. Protected-load rules, priority classes, and the relaxation order are encoded as an explicit, inspectable rule base (a small production-rule representation) rather than hard-coded branches in the solver, so the policy is versioned, editable, and arguable in a viva rather than only “trust the code.”

6.5 Fairness and excess-shedding metrics

Fairness is computed automatically at every replan cycle over cumulative shed-minutes per controllable feeder within a rolling window (Gini coefficient and max-to-mean ratio), and separately over interrupted energy (kWh), since a 2 MW feeder shed for ten minutes is not equivalent in impact to a 200 kW feeder shed for the same duration; it is reported as a diagnostic guardrail metric rather than a constraint the solver must satisfy exactly. Alongside it, an excess-shedding metric — actual shed load minus the required cut, aggregated per replan cycle and per

scenario tier — directly quantifies the over-shedding failure mode described in Section 1 and is tracked as a first-class evaluation output in its own right.

6.6 LLM explanation layer

The explanation layer answers operator queries (e.g., "why was Feeder 12 cut before Feeder 9?") strictly by querying the solver's constraint trace and relaxation log; it never sees or influences the decision. Its boundary is demonstrated explicitly by constructing queries the trace cannot answer and confirming the layer declines rather than fabricates a justification, consistent with the grounded-generation practice reviewed in Section 4.1.

7. DATASET / DATA COLLECTION

No public dataset records feeder-level cut/restore decisions for Indian DISCOMs, so this project does not claim to have or reconstruct one. Instead, real data is used to calibrate a synthetic-but-grounded grid, and the distinction between the two is made explicit throughout the report rather than blurred.

Source	Role	Status
Grid-India / POSOCO demand & shortage data	Calibrates realistic Indian demand magnitude and shortfall scale	Real (primary calibration)
CEA / data.gov.in peak-demand & power-supply data	Cross-validates the above at state level	Real (official cross-check)
CEA distribution-planning & feeder/DT reports	Grounds synthetic network scale and reliability assumptions	Real reference
IEEE / OpenDSS distribution test feeders (13-, 34-, 123-bus)	Grounds synthetic feeder topology structure	Real reference (non-Indian)
UCI ElectricityLoadDiagrams20112014	High-resolution (15-min) temporal load-shape reference	Real, non-Indian
Synthetic grid generator output	Feeder identities, priority classes, protected flags, load profiles, shedding history	Synthetic-but-grounded

Table 2. Real vs. synthetic data sources and their role in the pipeline.

The feeder model stores a time-varying load profile per feeder (timestamp, feeder_id, load_kw) rather than a single static value, since sheddable load depends on time of day. Feeder metadata (feeder_id, priority_class, protected, shed_allowed, max_consecutive_downtime, min_state_duration) is generated hierarchically: substation → feeder → service area; with an energy-balance sanity check against the calibrated aggregate demand at each timestamp. Shortfall scenarios (mild / moderate / crisis severity, required_shed_kw over time) are generated by calibrating band thresholds against the empirical Indian shortage distribution rather than by assuming arbitrary percentage cut-offs.

8. TOOLS AND TECHNOLOGIES

Category	Tools
Language	Python 3.11+
Solvers	Backtracking + AC-3, min-conflicts local search, A* baseline — all implemented from scratch
Data handling	pandas, numpy, pyarrow (Parquet for time-series)
Simulation	Custom grid-topology and shortfall-scenario generators
Forecast adapter	An existing, pre-built time-series forecasting model, wired in as a pluggable input source (not trained/researched in-house)
Visualisation	matplotlib
Explanation layer	Anthropic API, restricted to read-only queries against the constraint trace
EDA / documentation	Jupyter notebooks
Version control	Git / GitHub, with a develop branch and per-member feature branches

Table 3. Tools and technologies used across the project.

9. EXPECTED OUTCOMES

- A working, from-scratch CSP scheduler with two solving strategies, benchmarked against each other and against greedy and A* baselines on synthetic grids of increasing scale (10 / 50 / 200 / 500 feeders), reporting solve time, solution quality, and excess shedding relative to the required cut.
- A tested constraint-relaxation cascade (Section 6.4) with a verified relaxation order — downtime, then switching frequency, then non-critical priority — and a guarantee, demonstrated across all test cases, that the priority/protection rules for critical loads are never relaxed.
- A fairness-and-excess-shedding report (Section 6.5) — Gini coefficient / max-to-mean ratio on outage duration and interrupted energy, plus a measured excess-shedding metric — tracked automatically across every replan cycle of a simulated operating period, showing where a purely cost-minimising schedule would concentrate outages unfairly or over-shed relative to the required cut.
- A working rolling-horizon replanning loop, with a measured plan-thrashing rate per scenario tier and a documented mitigation (hysteresis / minimum-state-duration constraint).
- A live-mode demonstration in which the pluggable Forecast Adapter is switched from scripted scenarios to an existing forecasting model over public demand data, showing the full pipeline running end-to-end from raw demand signal to auditable schedule.

- A read-only LLM explanation layer with a verified boundary — shown, via a constructed test set, to be unable to answer beyond the logged constraint trace.
- A reproducible GitHub repository with documented data lineage (data-sources.md) and a final comparative report covering all of the above, defensible section-by-section by any team member in a viva.

10. SCOPE OF THE PROJECT

In scope

- A CSP-based decision layer, with an optimisation objective layered on top, that consumes a given shortfall and produces a safe, auditable cut schedule that minimises harm and unnecessary excess shedding.
- Two from-scratch CSP solvers, a constraint-relaxation cascade, a rolling-horizon control loop, fairness-and-excess-shedding metrics, and a read-only LLM explanation layer.
- A pluggable Forecast Adapter that integrates an existing, pre-built forecasting model as the shortfall input for a live-mode demo, alongside scripted scenarios for controlled benchmarking.
- An aggregated feeder-level decision model — not a full electrical simulation of power flow, voltage stability, or transformer limits.
- A synthetic-but-grounded distribution grid, explicitly calibrated against public Indian and reference datasets.
- A simulated actuation layer with confirmable success/failure states.

Out of scope

- Researching or training a new forecasting model — S_t is treated strictly as an input (see Section 2), and the project uses an existing model rather than developing forecasting capability of its own.
- Full electrical grid modelling (power flow, voltage stability, transformer limits) and production-grade SCADA / live grid integration — feeder states and electrical behaviour are simulated, not solved via power-flow equations or connected to real hardware.
- Sectionalised, sub-feeder-level control — the core project switches whole feeders; finer-grained control is noted as a possible future extension beyond this synopsis.
- Claims of real, historical, feeder-level load-shedding decisions — no such public dataset exists, and none is fabricated.

11. PROJECT TIMELINE

The 15-week plan below assumes a standard semester and is adjusted proportionally if the actual course calendar differs. Ownership reflects each member's primary role: Parth — CSP solvers, KR&R rule base, and overall architecture (team lead); Mitali — simulation, data pipeline, EDA, and evaluation; Mehak — literature review, forecast-adapter integration, LLM explanation layer, and project coordination.

#	Weeks	Deliverable	Exit Criteria
1	1–2	Formal CSP definition; related-work table; dataset selection	Formal spec approved by all 3; data-sources.md lists ≥ 4 verified sources
2	3–4	Synthetic grid-topology generator; shortfall-scenario generator	Generates valid, protected-tagged topologies at 10/50/200/500 feeders
3	5–6	Baseline CSP solver (backtracking + AC-3)	100% constraint satisfaction at ≤ 50 feeders on all test scenarios
4	7–8	Min-conflicts solver, greedy baseline, first comparison	Comparison table (solve time + harm-cost) across 10/50/200 feeders
5	9	Rolling-horizon replanning loop	Replans correctly on cadence + shortfall-delta triggers
6	10	Constraint-relaxation cascade	Relaxation fires only when infeasible; priority/protection rules never relaxed
7	11	A* baseline + fairness/excess-shedding metrics wired into replan loop	A* labelled non-comparable; Gini/max-to-mean and excess-shedding logged every cycle
8	12	Forecast Adapter integration (existing model, pluggable input)	Adapter swaps cleanly between scripted scenarios and live forecast output
9	13	Scalability sweep; thrashing test; relaxation report	Sweep run 10→500 feeders; thrashing and relaxation rates reported per tier
10	14	LLM explanation layer	Answers verified against trace; boundary test passes
11	15	Full comparative write-up; repo finalisation; viva prep	Repo runs clean from a fresh clone; draft covers all deliverables; each member can defend any section

Table 4. Eleven-phase implementation timeline with named ownership and exit criteria.

12. REFERENCES

[1] National Load Dispatch Centre (POSOCO), "Short-Term National Resource Adequacy Plan (ST-NRAP)," discussed in "High-Risk Load Shedding in India: Summer 2025 Power Cuts Loom," Down To Earth, Apr. 2025.

[2] "India to Crack Down on DISCOM Load Shedding," Mercom India, Dec. 2017.

[3] "Electricity utilities warned of 'consequences' over excuses for loadshedding," The Asian Age, reprinted via PressReader, Apr. 2015 (rotational load-shedding directive covering slum and JJ-cluster areas).

[4] D. Bienstock and A. Verma, "Worst-Case Load Shedding in Electric Power Networks," arXiv:1712.00536, 2017.

[5] "Scalable Learning for Optimal Load Shedding Under Power Grid Emergency Operations," arXiv:2111.11980, 2021.

[6] S. Minton, M. D. Johnston, A. B. Philips, and P. Laird, “Minimizing Conflicts: A Heuristic Repair Method for Constraint Satisfaction and Scheduling Problems,” Artificial Intelligence, vol. 58, pp. 161–205, 1992.

[7] “Logic Programming Approaches for Representing and Solving Constraint Satisfaction Problems: A Comparison,” arXiv:cs/0011030 (Belgian electricity-utility maintenance-scheduling case study).

[8] “Equity-aware Load Shedding Optimization,” arXiv:2406.17877, 2024.

[9] G. T. Costa Oliveira et al., “Use of Model Predictive Control for Experimental Microgrid Optimization,” Applied Energy, ScienceDirect, 2014.

[10] “A Rolling Horizon Approach for Optimal Management of Microgrids Under Stochastic Uncertainty,” Energy Conversion and Management, ScienceDirect.

[11] O. Ayala and P. Bechard, “Reducing Hallucination in Structured Outputs via Retrieval-Augmented Generation,” Proc. NAACL Industry Track, 2024.

[12] “Exploring RAG Solutions to Reduce Hallucinations in LLMs,” IEEE Conference Publication, 2025.

[13] C. Niu et al., “RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models,” Proc. ACL, pp. 10862–10878, 2024.

[14] Central Electricity Authority, Government of India, “Electricity Distribution Network Planning Criteria” and feeder/DT metering reports, cea.nic.in.

[15] Open Government Data Platform India, “Monthly Peak Demand for Power” and “Power Supply Position” catalogs, data.gov.in.

[16] UCI Machine Learning Repository, “ElectricityLoadDiagrams20112014 Data Set,” archive.ics.uci.edu (CC BY 4.0).

[17] “The Art of Load Shedding and Online Applications in a Power System Under an Emergency State,” Electrical Engineering Portal (EEP), electrical-engineering-portal.com (defines and discusses over-shedding in staged load-shedding schemes).